Ch 5.1.5: k-fold Cross-Validation for Classification

Lecture ~~12~~ 13 - CMSE 381

Prof. Elizabeth Munch

Michigan State University
::
Dept of Computational Mathematics, Science & Engineering

Mon, Oct 10, 2022

Announcements

Last time:

- k-fold CV

This lecture:

- CV for classification

Announcements:

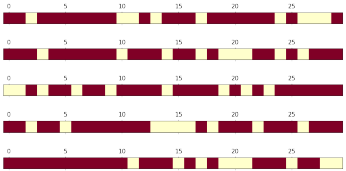
- Homework #4 is posted, Due tonight
-

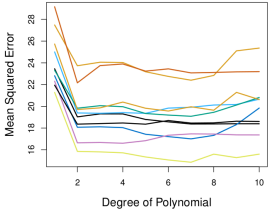
Section 1

Last time

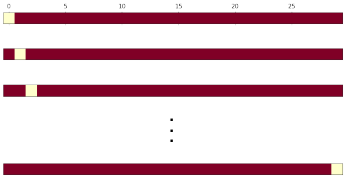
Approximations of Test Error

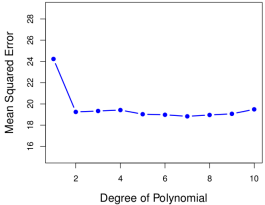
Validation Set



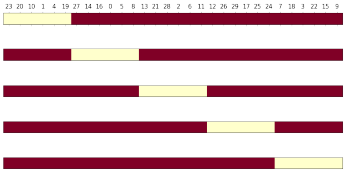


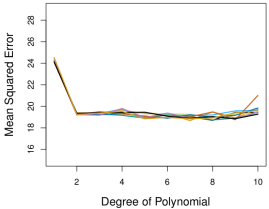
LOOCV





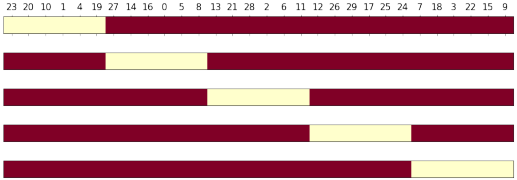
K-fold CV





Definition of k-fold CV – Regression

- Randomly split data into k-groups (folds)
- Approximately equal sized. For the sake of notation, say each set has ℓ points



- Remove ith fold U_i and reserve for testing.
- Train the model on remaining points
- Calculate $\mathrm{MSE}_i = \frac{1}{\ell}\sum_{(x_j,y_j)\in U_i}(y_j - \hat{y}_j)^2$
- Rinse and repeat

Return

$$CV_{(k)} = \frac{1}{k}\sum_{i=1}^{k}\mathrm{MSE}_i$$

Section 2

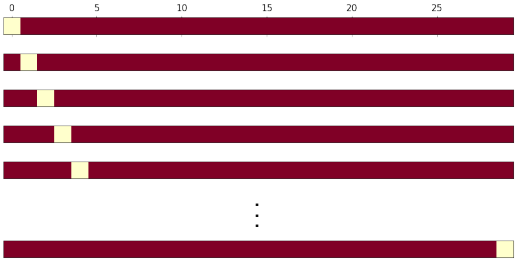
CV for Classification

Setup: LOOCV

- Remove ith point (x_i, y_i) and reserve for testing.
- Train the model on remaining points
- Calculate $\mathrm{Err}_i = \underbrace{\mathrm{I}(y_j \neq \hat{y}_j)}$

$$= \begin{cases} 1 & \text{incorrect} \quad y_j \neq \hat{y}_j \\ 0 & \text{correct} \end{cases}$$

- Rinse and repeat



Return

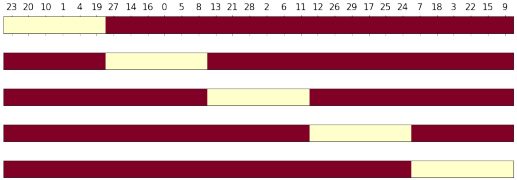
$$CV_{(n)} = \frac{1}{n}\sum_{i=1}^{n} \mathrm{Err}_i$$

$$= \frac{\#\ \text{incorrect "models"}}{n}$$

Setup: k-fold

Classification!

- Randomly split data into k-groups (folds)
- Approximately equal sized. For the sake of notation, say each set has ℓ points



- Remove ith fold U_i and reserve for testing.
- Train the model on remaining points
- Calculate $\underbrace{\mathrm{Err}_i = \frac{1}{\ell}\sum_{(x_j,y_j)\in U_i} \mathrm{I}(y_j \neq \hat{y}_j)}$ $= \dfrac{\#\text{ times wrong on that test set}}{\#\text{ pts in test set}}$
- Rinse and repeat

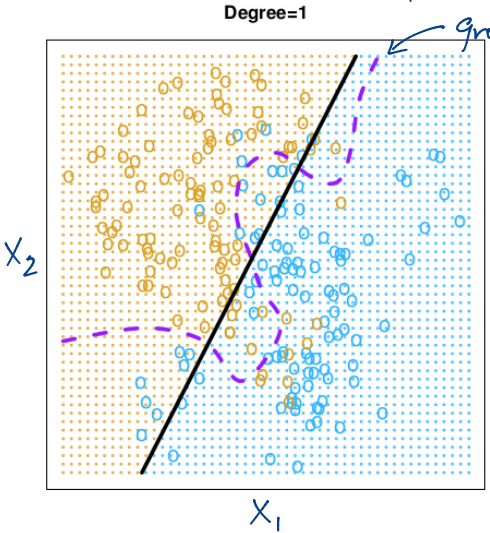
Return

$$CV_{(k)} = \frac{1}{k}\sum_{i=1}^{k} \underline{\underline{\mathrm{Err}_i}}$$

Example on simulated data: Linear

Y = color ∈ {blue / yellow}

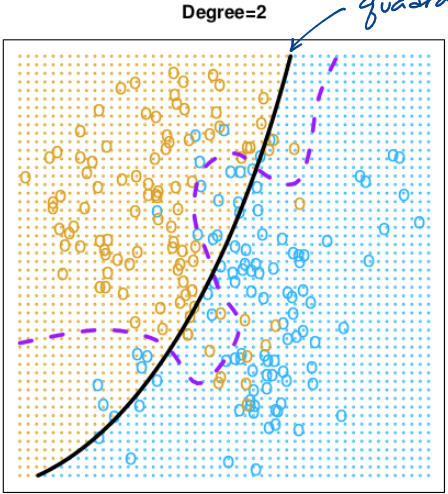
grand truth



- Purple: Bayes decision boundary.
  - Error rate: 0.133
- Black: Logistic regression
  - $\log(p/(1-p)) = \beta_0 + \beta_1 X_1 + \beta_2 X_2$
  - Error rate: 0.201

LOOCV

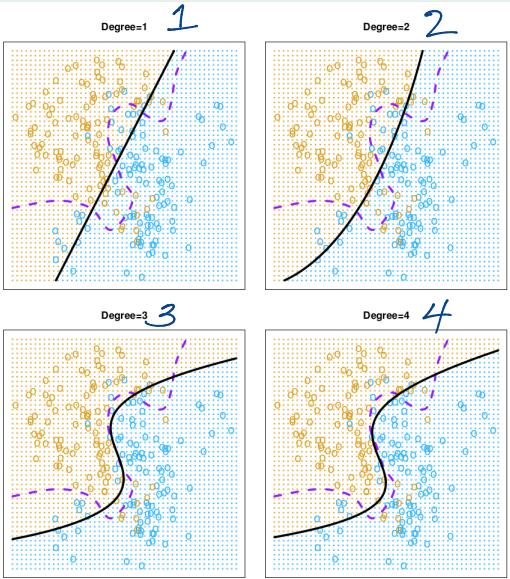
Example on simulated data: Quadratic logistic regression



quadratic learned model

- Purple: Bayes decision boundary.
  - Error rate: 0.133
- Black: Logistic regression
  - $\log(p/(1-p)) = \beta_0 + \beta_1 X_1 + \beta_2 X_1^2 + \beta_3 X_2 + \beta_4 X_2^2$ $(X_2)^2$
  - Error rate: 0.197

Example on simulated data: all the polynomials!

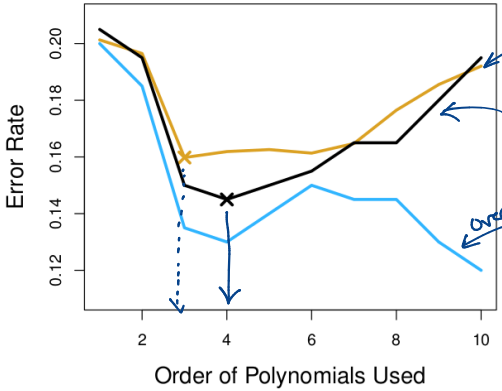


- Purple: Bayes decision boundary.
  - Error rate: 0.133

Irreducible error

- Black: Logistic regression
  - Deg 1 Error rate: 0.201
  - Deg 2 Error rate: 0.197
  - Deg 3 Error rate: 0.160
  - Deg 4 Error rate: 0.162

Decide degree based on CV



ground truth

10-fold CV test error approximation

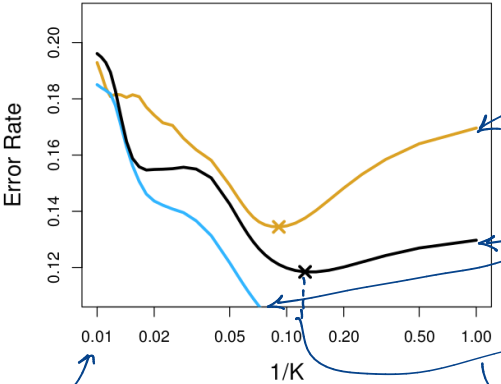
overfitting

- Test error (brown)
- Training error (blue)
- 10-fold CV error (black)

↳ U-shaped like before

bias/variance tradeoff

Similar game for KNN



- Test error (brown)
- Training error (blue)
- 10-fold CV error (black)

$\frac{1}{K} = 0.12$

$K = 1$

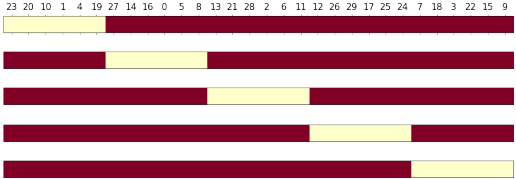
$K = 100$ nearest neighbors

KNN k # of neighbors

Coding - k-fold for penguin classification section

TL;DR

k-fold CV



$$CV_{(k)} = \frac{1}{k}\sum_{i=1}^{k} \mathrm{MSE}_i$$

Use $k = 5$ or 10 usually

k-fold CV for classification

$$\mathrm{Err}_i = \mathrm{I}(y_j \neq \hat{y}_j)$$

$$CV_{(k)} = \frac{1}{k}\sum_{i=1}^{k} \mathrm{Err}_i$$

Next time

10	M	Oct 3	Leave one out CV	5.1.1, 5.1.2	
11	W	Oct 5	k-fold CV	5.1.3	
12	F	Oct 7	More k-fold CV,	5.1.4-5	
13	M	Oct 10	k-fold CV for classification	5.1.5	HW #4 Due
14	W	Oct 12	Resampling methods: Bootstrap	5.2	
15	F	Oct 14	Subset selection	6.1	
16	M	Oct 17	Shrinkage: Ridge	6.2.1	HW #5 Due
17	W	Oct 19	Shrinkage: Lasso	6.2.2	
18	F	Oct 21	[No class, Dr Munch out of town]		
	M	Oct 24	No class - Fall break		
19	W	Oct 26	Dimension Reduction	6.3	
20	F	Oct 28	More dimension reduction; High dimensions	6.4	HW #6 Due
	M	Oct 31	Review		
	W	Nov 2	***Midterm #2***		

* Do not use cross_val_score for homework! *

train

test

train model

model.predict_proba(test_set)

$\beta_0 \beta_1, \beta_2 \ldots \beta_3$

yhat = model.predict(test_set)

model = LogisticRegression()

model.fit(X, y)

$$\begin{bmatrix} \text{red} \\ \text{blue} \\ \text{blue} \\ \text{red} \\ \text{red} \\ \vdots \end{bmatrix}$$